ASSIGNMENT-11.5

H.no-2203A51377

Batch – 30

Task 1 – Stack Implementation

Prompt

Design and implement a Stack data structure supporting basic stack operations. A Python Stack class supporting push, pop, peek, and is_empty operations with proper documentation.

CODE

Lab_exam.py | Assignmnet_10.2.py | Assignmnet_13.1.py | Assignment_13.5.py | Assignment_11.5.py | Assignment_9.5.py

Assignment_11.5.py > ...

```python
#Design and implement a Stack data structure supporting basic stack operations. A Python Stack class supporting push, po
class Stack:
    """A simple implementation of a Stack data structure."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []  # Use a list to store stack items

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)  # Append item to the end of the list

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack")  # Raise error if stack is empty
        return self.items.pop()  # Remove and return the last item in the list

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack")  # Raise error if stack is empty
        return self.items[-1]  # Return the last item in the list

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.items) == 0  # Check if the list is empty
# Example usage:
stack = Stack()  # Create a new stack instance
stack.push(1)  # Push an item onto the stack
stack.push(2)  # Push another item onto the stack
print(stack.peek())  # Output: 2 (the top item)
print(stack.pop())  # Output: 2 (removes the top item)
print(stack.is_empty())  # Output: False (stack is not empty)
print(stack.pop())  # Output: 1 (removes the last item)
print(stack.is_empty())  # Output: True (stack is now empty)
```

OUTPUT

```
2
2
False
1
True
```

JUSTIFICATION

A stack is a linear data structure that follows the Last In First Out (LIFO) principle. The element inserted last is removed first. Stacks are widely used in function calls, recursion, undo operations, and expression evaluation in compilers.

Task 2 – Queue Implementation

Prompt

Create a Queue data structure following FIFO principles. A complete Queue implementation including enqueue, dequeue, front element access, and size calculation.

CODE

```python
#Create a Queue data structure following FIFO principles. A complete Queue implementation including enqueue, dequ
class Queue:
    """A simple implementation of a Queue data structure following FIFO principles."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []  # Use a list to store queue items

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)  # Append item to the end of the list

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")  # Raise error if queue is empty
        return self.items.pop(0)  # Remove and return the first item in the list

    def front(self):
        """Return the item at the front of the queue without removing it. Raises an error if the queue is empty."
        if self.is_empty():
            raise IndexError("Front from an empty queue")  # Raise error if queue is empty
        return self.items[0]  # Return the first item in the list

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)  # Return the length of the list
# Example usage:
queue = Queue()  # Create a new queue instance
queue.enqueue(1)  # Enqueue an item
queue.enqueue(2)  # Enqueue another item
print(queue.front())  # Output: 1 (the front item)

print(queue.dequeue())  # Output: 1 (removes the front item)
print(queue.size())  # Output: 1 (one item left in the queue)
print(queue.dequeue())  # Output: 2 (removes the last item)
print(queue.is_empty())  # Output: True (queue is now empty)
```

OUTPUT

```
1
1
1
False
2
```

JUSTIFICATION

A queue follows the First In First Out (FIFO) principle where the first inserted element is removed first. Queues are commonly used in CPU scheduling, printer task management, and network data handling where processing order must be maintained.

Task 3 – Singly Linked List

Prompt

Build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality.

CODE

```python

#Build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node
class Node:
    """A class representing a node in a singly linked list."""
    def __init__(self, data):
        """Initialize a node with data and a pointer to the next node."""
        self.data = data  # Store the data in the node
        self.next = None  # Initialize the next pointer to None
class SinglyLinkedList:
    """A class representing a singly linked list."""
    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None  # Initialize the head of the list to None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)  # Create a new node with the provided data
        if self.head is None:
            self.head = new_node  # If the list is empty, set the new node as the head
            return
        last_node = self.head  # Start from the head of the list
        while last_node.next:  # Traverse to the end of the list
            last_node = last_node.next
        last_node.next = new_node  # Link the last node to the new node

    def display(self):
        """Display all nodes in the linked list."""
        current_node = self.head  # Start from the head of the list
        while current_node:  # Traverse through the list until the end
            print(current_node.data)  # Print the data of the current node
            current_node = current_node.next  # Move to the next node
# Example usage:
linked_list = SinglyLinkedList()  # Create a new singly linked list instance

linked_list.insert(1)  # Insert a node with data 1
linked_list.insert(2)  # Insert a node with data 2
linked_list.insert(3)  # Insert a node with data 3
linked_list.display()  # Display the linked list (Output: 1, 2, 3)
```

OUTPUT

```
1
2
3
```

JUSTIFICATION

A singly linked list is a dynamic data structure where elements are stored in nodes connected through pointers. It allows efficient insertion and deletion operations. Linked lists are widely used in implementing stacks, queues, and memory management systems.

Task 4 – Hash Table Implementation

Prompt

Create a hash table with collision handling. Hash table supporting insert, search, and delete operations using chaining.

CODE

```python

#Create a hash table with collision handling. Hash table supporting insert, search, and delete ope
class HashTable:
    """A simple implementation of a hash table with collision handling using chaining."""
    def __init__(self, size=10):
        """Initialize the hash table with a specified size."""
        self.size = size  # Set the size of the hash table
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining
    def _hash(self, key):

        """Generate a hash for the given key."""
        return hash(key) % self.size  # Use built-in hash function and mod by size to get index
    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)  # Get the index for the key
        for i, (k, v) in enumerate(self.table[index]):  # Check for existing key in the chain
            if k == key:
                self.table[index][i] = (key, value)  # Update value if key already exists
                return
        self.table[index].append((key, value))  # Add new key-value pair to the chain
    def search(self, key):
        """Search for a value by key in the hash table."""
        index = self._hash(key)  # Get the index for the key
        for k, v in self.table[index]:  # Search through the chain at the index
            if k == key:
                return v  # Return the value if key is found
        return None  # Return None if key is not found
    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)  # Get the index for the key
        for i, (k, v) in enumerate(self.table[index]):  # Search through the chain at the index
            if k == key:
                del self.table[index][i]  # Remove the key-value pair if key is found
                return
# Example usage:
hash_table = HashTable()  # Create a new hash table instance
hash_table.insert("name", "Alice")  # Insert a key-value pair into the hash table
hash_table.insert("age", 30)  # Insert another key-value pair into the hash
print(hash_table.search("name"))  # Output: Alice (search for a key in the hash table)
print(hash_table.search("age"))  # Output: 30 (search for another key in the hash table)
hash_table.delete("name")  # Delete a key-value pair from the hash table
print(hash_table.search("name"))  # Output: None (search for a deleted key in the hash table)

```

OUTPUT

```
Alice
30
None
```

JUSTIFICATION

A hash table stores data using key-value mapping, allowing extremely fast data access. The average time complexity for searching is O(1). Hash tables are widely used in databases, dictionaries, caching systems, and indexing.

Task 5 – Graph Representation

Prompt

Implement a graph using an adjacency list representation with methods to add vertices, add edges, and display connections.

CODE

```python


#Implement a graph using an adjacency list representation with methods to add vertices, add edges, and display connectio
class Graph:
    """A simple implementation of a graph using an adjacency list representation."""
    def __init__(self):
        """Initialize an empty graph."""
        self.graph = {}  # Use a dictionary to store the adjacency list

    def add_vertex(self, vertex):
        """Add a vertex to the graph."""
        if vertex not in self.graph:
            self.graph[vertex] = []  # Initialize an empty list for the vertex's edges

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices in the graph."""
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)  # Add vertex2 to the adjacency list of vertex1
            self.graph[vertex2].append(vertex1)  # Add vertex1 to the adjacency list of vertex2 (for undirected graph)

    def display(self):
        """Display the connections in the graph."""
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {', '.join(edges)}")  # Print each vertex and its connected edges
# Example usage:
graph = Graph()  # Create a new graph instance
graph.add_vertex("A")  # Add vertex A to the graph
graph.add_vertex("B")  # Add vertex B to the graph
graph.add_vertex("C")  # Add vertex C to the graph
graph.add_edge("A", "B")  # Add an edge between vertex A and vertex B
graph.add_edge("A", "C")  # Add an edge between vertex A and vertex C
graph.display()  # Display the connections in the graph (Output: A: B, C; B: A; C: A)

```

OUTPUT

```
A: B, C
B: A
C: A
```

JUSTIFICATION

Graphs are used to represent relationships between objects using vertices and edges. The adjacency list representation is efficient for storing sparse graphs. Graphs are widely used in network routing, social networks, and map navigation systems.and supports scalable validation logic.

Task 6 – Smart Hospital Management System

Prompt

Design a Smart Hospital Management System by selecting appropriate data structures for different hospital operations such as patient check-in, emergency case handling, medical records storage, appointment scheduling, and hospital room navigation.

CODE

```python

#Design a Smart Hospital Management System by selecting appropriate data structures for different hospital operations such as patient c
class HospitalManagementSystem:

    """A simple implementation of a Smart Hospital Management System."""
    def __init__(self):
        """Initialize the hospital management system with necessary data structures."""
        self.patients = {}  # Use a dictionary to store patient information
        self.emergency_cases = []  # Use a list to manage emergency cases
        self.medical_records = {}  # Use a dictionary to store medical records
        self.appointments = []  # Use a list to manage appointments
        self.rooms = {}  # Use a dictionary to manage hospital rooms

    def check_in_patient(self, patient_id, patient_info):
        """Check in a patient and store their information."""
        self.patients[patient_id] = patient_info  # Store patient information in the dictionary

    def handle_emergency_case(self, case_info):
        """Handle an emergency case by adding it to the emergency cases list."""
        self.emergency_cases.append(case_info)  # Add emergency case information to the list

    def store_medical_record(self, patient_id, record):
        """Store a medical record for a patient."""
        if patient_id not in self.medical_records:
            self.medical_records[patient_id] = []  # Initialize a list for the patient's medical records
        self.medical_records[patient_id].append(record)  # Add the medical record to the patient's list

    def schedule_appointment(self, appointment_info):
        """Schedule an appointment by adding it to the appointments list."""
        self.appointments.append(appointment_info)  # Add appointment information to the list

    def navigate_to_room(self, room_number):
        """Navigate to a hospital room by checking if it exists in the rooms dictionary."""
        if room_number in self.rooms:
            return f"Navigating to room {room_number}"  # Return navigation message if room exists
        else:
            return f"Room {room_number} does not exist"  # Return error message if room does not exist
# Example usage:
hospital = HospitalManagementSystem()  # Create a new instance of the hospital management system
hospital.check_in_patient("P001", {"name": "John Doe", "age":
30})  # Check in a patient with ID P001
hospital.handle_emergency_case({"case_id": "E001", "description": "Heart Attack"})  # Handle an emergency case
hospital.store_medical_record("P001", {"record_id": "R001", "details                                                    ": "Blood P
hospital.schedule_appointment({"appointment_id": "A001", "patient_id": "P001", "date": "2024-07-01"})  # Schedule an appointment for pa
hospital.rooms["101"] = "General Ward"  # Add a room to the hospital
print(hospital.navigate_to_room("101"))  # Navigate to an existing room (Output: Navigating to room 101)
print(hospital.navigate_to_room("102"))  # Navigate to a non-existing room (Output: Room 102 does not exist)
```

```
# table mapping feature → chosen data structure → justification.
# | Feature                | Chosen Data Structure | Justification                                                                |
# |------------------------|----------------------|------------------------------------------------------------------------------|
# | Patient Check-in       | Dictionary           | Allows for efficient storage and retrieval of patient information using unique patient IDs as keys. |
# | Emergency Case Handling | List                | Maintains an ordered collection of emergency cases, allowing
# | Medical Records Storage | Dictionary          | Enables efficient storage and retrieval of medical records associated with patient IDs. |
# | Appointment Scheduling  | List                | Maintains an ordered collection of appointments, allowing for easy addition and retrieval. |
# | Hospital Room Navigation | Dictionary          | Allows for efficient storage and retrieval of room information using room numbers as keys. |
```

OUTPUT

```
Navigating to room 101
Room 102 does not exist
```

JUSTIFICATION

Queue is suitable for patient check-in because patients are treated in the order they arrive (FIFO). This ensures fairness and systematic patient handling in hospitals. Queues are commonly used in service management systems such as hospitals, banks, and ticket counters.

Task 7 – Smart City Traffic Control System

Prompt

Design a Smart Traffic Management System by selecting suitable data structures for traffic signal queues, emergency vehicle handling, vehicle registration lookup, road network mapping, and parking slot tracking.

CODE

```python
#Design a Smart Traffic Management System by selecting suitable data structures for traffic signal queues, emergency vehicle handling, vehicle registration lookup, road network
import numbers

class TrafficManagementSystem:
    """A simple implementation of a Smart Traffic Management System."""
    def __init__(self):
        """Initialize the traffic management system with necessary data structures."""
        self.traffic_signal_queues = {}  # Use a dictionary to manage traffic signal queues
        self.emergency_vehicle_queue = []  # Use a list to manage emergency vehicle queue
        self.vehicle_registration_lookup = {}  # Use a dictionary for vehicle registration lookup
        self.road_network_mapping = {}  # Use a dictionary to represent the road network
        self.parking_slot_tracking = {}  # Use a dictionary to track parking slots
    # Additional methods for managing traffic signals, emergency vehicles, vehicle registration, road network, and parking slots would be implemented here
# Example usage:
traffic_system = TrafficManagementSystem()  # Create a new instance of the traffic management system
# The following lines would demonstrate the usage of the traffic management system's features, such as managing

# traffic signal queues, handling emergency vehicles, looking up vehicle registrations, mapping the road network, and tracking parking slots.
# Table mapping feature → chosen data structure → justification.
# | Feature                    | Chosen Data Structure | Justification                                                                  |
# |----------------------------|----------------------|--------------------------------------------------------------------------------|
# | Traffic Signal Queues      | Dictionary           | Allows for efficient management of multiple traffic signal
# | Emergency Vehicle Handling  | List                 | Maintains an ordered collection of emergency vehicles, allowing for easy addition and retrieval. |
# | Vehicle Registration Lookup | Dictionary           | Enables efficient storage and retrieval of vehicle registration information using license plate numbers as keys. |
# | Road Network Mapping        | Dictionary           | Represents the road network as an adjacency list, allowing for efficient traversal and pathfinding. |
# | Parking Slot Tracking       | Dictionary           | Allows for efficient tracking of parking slot availability using slot numbers as keys. |
```

JUSTIFICATION

Queues are ideal for managing vehicles at traffic signals because vehicles pass in the order they arrive. This maintains smooth traffic flow and prevents unfair overtaking. Queue structures are commonly used in traffic systems and transportation management.

Task 8 – Smart E-Commerce Platform

Prompt

Design a Smart Online Shopping System by selecting suitable data structures for shopping cart management, order processing, product ranking, product search, and delivery route planning.

CODE

```python
#Design a Smart Online Shopping System by selecting suitable data structures for shopping cart management, order processing, product ranking, product search, and delivery route planni
class OnlineShoppingSystem:
    """A simple implementation of a Smart Online Shopping System."""
    def __init__(self):
        """Initialize the online shopping system with necessary data structures."""
        self.shopping_cart = []  # Use a list to manage the shopping cart
        self.orders = []  # Use a list to manage orders
        self.product_ranking = {}  # Use a dictionary to manage product rankings
        self.product_search = {}  # Use a dictionary to manage product search
        self.delivery_route_planning = {}  # Use a dictionary to manage delivery route planning
    # Additional methods for managing the shopping cart, processing orders, ranking products, searching products, and planning delivery routes would be implemented here
# Example usage:
shopping_system = OnlineShoppingSystem()  # Create a new instance of the online shopping system

# Table mapping feature → chosen data structure → justification.
# | Feature                  | Chosen Data Structure | Justification                                                                                |

# |-------------------------|----------------------|-----------------------------------------------------------------------------------------|
# | Shopping Cart Management | List                 | Maintains an ordered collection of items in the shopping cart, allowing for easy addition and removal of products. |
# | Order Processing         | List                 | Maintains an ordered collection of orders, allowing for easy tracking and management of customer orders. |
# | Product Ranking          | Dictionary           | Enables efficient storage and retrieval of product rankings using product IDs as keys. |
# | Product Search           | Dictionary           | Allows for efficient search of products using product names    or IDs as keys, enabling quick access to product information. |
# | Delivery Route Planning   | Dictionary           | Represents delivery routes as a graph, allowing for efficient traversal and optimization of delivery paths. |
```

JUSTIFICATION

Queue is suitable for order processing because orders should be processed in the same order they are placed (FIFO). This ensures fairness and proper order management. Queue-based systems are widely used in e-commerce platforms and transaction processing systems.